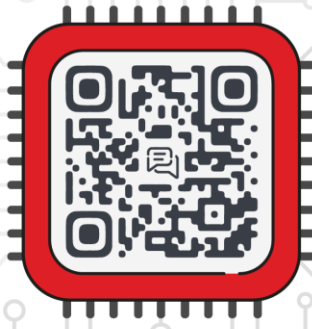




10

16

2400 sp

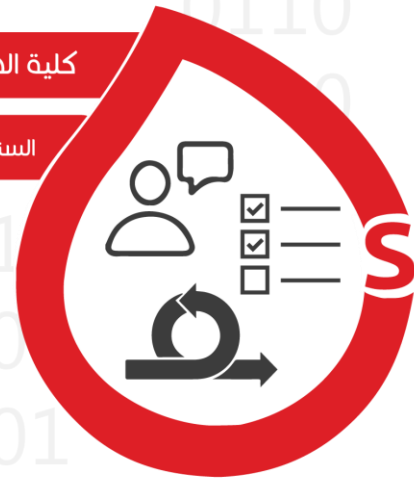
نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Quality Management

د. سعيد أبو طراب



RB Informatics; 10/06/2026

هندسة برمجيات ٢

مقدمة: قبل أن ندخل في إدارة الجودة، لو قمنا بتطوير نظام لإدارة مستشفى... النظام يعمل، جميع الأضرار تعمل، جميع العمليات الحسابية صحيحة والمتطلبات الموجودة في وثيقة الـ Requirements تم تنفيذها.

هل يمكن الحكم مباشرة ان النظام عالي الجودة؟

في الحقيقة... ليست بالضرورة لأن جودة البرمجيات ليست مجرد أن البرنامج يعمل، قد يعمل البرنامج بصورة صحيحة، ولكن:

- بطيء جداً.
- يتوقف كل فترة.
- يستهلك كمية هائلة من الذاكرة.
- يصعب تعديله.
- واجهته معقدة.

إذاً: نستنتج أنه قد يكون البرنامج صحيحاً (Correct)، لكنه ليس عالي الجودة (High Quality)

لماذا نحتاج إلى إدارة الجودة؟

نوضح ذلك من خلال مثال: تخيلوا شركة تطور نظاماً لبنك، النظام يحتوي على ملايين الأسطر البرمجية، يعمل عليه أكثر من 100 مبرمج و مدة المشروع سنتان.

السؤال الآن... إذا لم يكن هناك نظام واضح لضبط الجودة، ماذا سيحدث؟ سنواجه مشكلات مثل:

- كل مبرمج يكتب بأسلوب مختلف.
- لا توجد معايير ثابتة.
- الأخطاء تزداد مع الوقت.
- يصبح تعديل النظام صعباً.
- تكلفة الإصلاح ترتفع بشكل كبير.

لذلك ظهرت إدارة الجودة حتى تجعل عملية التطوير نفسها منظمة، وليس المنتج فقط.



الفرق بين Quality و Correctness

■ أولاً: صحة البرنامج (Correctness) : أي تنفيذ البرنامج ما هو مطلوب منه

مثال: جمع رقمين إذا أعطيناها: 7 + 5 وأعاد: 12 إذن البرنامج صحيح، لكن... هل هو عالي الجودة؟ ليس بالضرورة.

■ ثانياً: الجودة (Quality) هي مجموعة الخصائص والصفات التي تجعل المنتج البرمجي ذا قيمة بالنسبة لجميع الأطراف المعنية، بحيث يلبي متطلبات المستخدم وتوقعاته، ويحقق أهداف صاحب المنتج، ويكون سهل التطوير والصيانة بالنسبة للمطور.

الجودة من وجهات النظر المختلفة

تنظر كل جهة إلى الجودة بطريقة مختلفة، لذلك يختلف مفهوم الجودة باختلاف الشخص الذي ينظر إلى المنتج:
من وجهة نظر المستخدم: يهتم بأن يلبي النظام احتياجاته، وأن يكون سهل الاستخدام، وسعره مناسباً، وقابلاً للتطوير مستقبلاً.

من وجهة نظر صاحب المنتج: يهتم بأن يكون المنتج قابلاً للبيع على نطاق واسع، قليل التكلفة، وسهل الصيانة والدعم.
من وجهة نظر المطور: يهتم بأن يكون الكود منظماً (Clean Code)، وأن يعتمد على معمارية واضحة تسهل تطويره وصيانه.

ولأن مفهوم الجودة يختلف من منظور إلى آخر، فلا يوجد تعريف واحد شامل لها، وإنما يمكن النظر إليها من أربعة جوانب رئيسية:

التعريف الأول: مدى تحقيق المنتج أو النظام للمتطلبات المحددة مسبقاً.

أي أن لدينا وثيقة Requirements، فإذا تم تنفيذ جميع المتطلبات الواردة فيها بالشكل المطلوب، فإن النظام يحقق مستوى جيداً من الجودة إلا أن هذا التعريف يركز فقط على ما هو مكتوب في وثيقة المتطلبات.

التعريف الثاني: مدى تحقيق النظام لاحتياجات المستخدم وتوقعاته.

فالمستخدم لا يقرأ وثيقة المتطلبات، وإنما يحكم على جودة النظام من خلال تجربته أثناء الاستخدام لذلك قد يكون النظام مطابقاً للمواصفات، ومع ذلك يشعر المستخدم بأنه غير مريح أو لا يلبي توقعاته، فتكون الجودة منخفضة من وجهة نظره.

التعريف الثالث: مطابقة جميع المتطلبات الوظيفية (Functional Requirements) ومتطلبات الأداء (Performance Requirements).

أي أن النظام لا يقتصر على تنفيذ الوظائف المطلوبة، بل يؤديها أيضاً بالكفاءة ومستوى الأداء المطلوبين.

التعريف الرابع: امتلاك النظام للخصائص الضمنية (Implicit Characteristics) المتوقعة في أي برنامج احترافي، حتى وإن لم تذكر صراحةً في وثيقة المتطلبات.

ومن أهم هذه الخصائص:

■ القابلية للصيانة

■ الاعتمادية (Reliability)

■ سهولة الاستخدام

■ القابلية للتوسع (Scalability)

وهذه تسمى: **المتطلبات غير الوظيفية** (Non-Functional Requirements)

لماذا تعتبر جودة البرمجيات مشكلة معقدة؟

- قد يبدو الأمر بسيطاً للوهلة الأولى، كتابة المتطلبات وتنفيذها ومن ثم اختبار النظام ثم الانتهاء، لكن الواقع مختلف تماماً لأن البرمجيات تختلف عن المنتجات المادية، فعندما تصنع كرسيًا مثلاً، يمكنك قياس طولهِ وعرضهِ وقوة تحمليه بسهولة.
- أما في البرمجيات فإن قياس سهولة الاستخدام وقابلية الصيانة وجودة تصميم النظام كلها أشياء يصعب قياسها مباشرة ولهذا السبب فإن إدارة الجودة في البرمجيات أصعب بكثير من إدارة الجودة في الصناعات التقليدية.

المشكلة الأولى: اختلاف وجهة نظر العميل والمطور أي أن هناك توتراً (Tension) بين ما يريده العميل وما يهتم به المطور.

العميل يهتم بـ:

- السرعة.
- الاعتمادية (Reliability)
- سهولة الاستخدام (Usability)
- الاستقرار.

باختصار، لا يهتم العميل بطريقة كتابة الكود، وإنما يهتم بما يراه ويستخدمه في النظام.

أما المطور يهتم بـ:

- القابلية للصيانة (Maintainability)
- إعادة الاستخدام (Reusability)
- وضوح الكود.
- سهولة الاختبار.
- قابلية التوسع.

المفترض وجود توازن بين احتياجات العميل واحتياجات فريق التطوير

المشكلة الثانية: بعض متطلبات الجودة يصعب تحديدها بدقة

مثال: كتب العميل: "واجهة النظام يجب أن تكون سهلة"، سيأتي السؤال مباشرة: سهلة بالنسبة لمن هل للطالب أم المهندس أم للمستخدم العادي لأن كل شخص قد يفسر كلمة "سهلة" بطريقة مختلفة.

ولهذا السبب يفضل دائماً تحويل المتطلبات العامة إلى متطلبات قابلة للقياس.

المشكلة الثالثة: المواصفات تكون غالباً :

- غير مكتملة (Incomplete) : أي أن بعض الحالات لم يتم ذكرها
- وأحياناً غير متناسقة (Inconsistent) : وجود متطلبين قد يتعارضان مع بعضهما.

⚡ خصائص الجودة (Quality Attributes)

- الاعتمادية (Reliability) تقيس مدى استقرار النظام.
- الأداء (Performance) يقيس سرعة التنفيذ.
- القابلية للصيانة (Maintainability) تقيس سهولة تعديل النظام.
- القابلية للتوسع (Scalability) تقيس قدرة النظام على التعامل مع زيادة عدد المستخدمين.

يلخص الجدول التالي أبرز خصائص ومحددات الجودة (Quality Attributes) التي يعتمد عليها تقييم كفاءة وموثوقية الأنظمة البرمجية المختلفة.

Safety	Understandability	Portability
Security	Testability	Usability
Reliability	Adaptability	Reusability
Resilience	Modularity	Efficiency
Robustness	Complexity	Learnability

لا يمكن تحسين جميع الخصائص معاً لأنه: لو كنت مدير مشروع، وجاء العميل وقال أريد النظام:

الأرخص تكلفة

الأكثر أماناً

الأسرع

والأسرع في التطوير

الأقل استهلاكاً للذاكرة

الأسهل في الصيانة

نلاحظ انه لا يمكن تحقيق كل ذلك لأن بعض خصائص الجودة تتعارض مع بعضها.

مثال: زيادة المتانة (Robustness) قد تؤدي إلى انخفاض الأداء. (Performance)

التوضيح: لنفترض أننا نطور نظاماً بنكياً وقبل تنفيذ أي عملية تحويل، يقوم النظام بما يلي:

- التحقق من هوية المستخدم.
- التحقق من الرصيد.
- التحقق من صلاحية البطاقات.
- التحقق من وجود احتيال.
- تسجيل العملية في ملفات السجل (Logs).

كل هذه العمليات تزيد من موثوقية النظام لكنها تحتاج إلى وقت أي أن النظام أصبح أكثر أماناً واعتمادية.. ولكنه أصبح أبطأ قليلاً وبالتالي حصل تعارض.

خطة الجودة (Quality Plan) : هي وثيقة تحدد مسبقاً خصائص الجودة الأكثر أهمية وكيفية قياسها والتأكد من تحقيقها والأنشطة التي سنستخدمها أثناء المشروع ووجود الخطة يجعل جميع أعضاء الفريق يعملون نحو نفس الأهداف.

❧ كيف نقيس الجودة؟

من خلال وجود عملية تقييم الجودة (Quality Assessment Process)

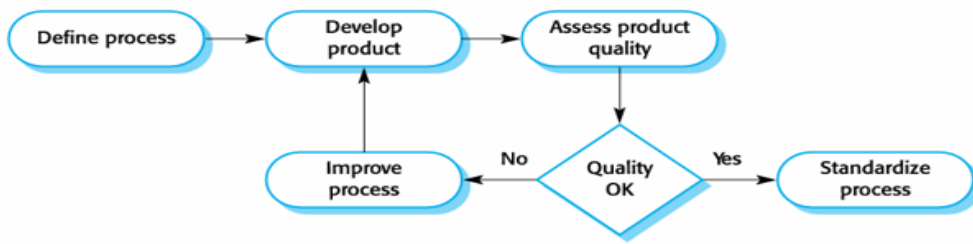
مثال:

- النظام سريع ثبت ذلك من خلال تحديد مقياساً مثل: متوسط زمن الاستجابة أقل من ثانيتين.
- النظام موثوق قد نقيس ذلك بعدد الأعطال خلال ألف ساعة تشغيل.
- النظام سهل الاستخدام قد نقيسه من خلال عدد الأخطاء التي يرتكبها المستخدم أثناء تنفيذ مهمة معينة، أو الوقت اللازم لإنجازها.
- إذن الجودة لا تترك للانطباعات، بل تحوّل إلى مؤشرات قابلة للقياس.

❧ العلاقة بين جودة المنتج وعملية التطوير

جودة المنتج تتأثر بجودة عملية التطوير أي إذا كانت طريقة العمل داخل الفريق فوضوية، فمن الصعب أن يكون المنتج النهائي عالي الجودة.

- في مرحلة ال plan يجب تحديد تعريف وما هي الصفات الواجب تحقيقها في المنتج لمعرفة عملية تقييم الجودة وبشكل عام فانه بمجرد اتباع process model معين فذلك يساعد لأنه في النتيجة سيكون المنتج ذو جودة كونه ينفذ وفق نموذج معين، ومع ذلك فان ادارة الجودة تتم وفق الالية كالتالي



ملاحظة: العلاقة بين عملية التطوير وجودة المنتج ليست علاقة مباشرة أو بسيطة لأن جودة المنتج لا تعتمد على عملية التطوير فقط بل تتأثر أيضاً ب: خبرة الفريق، تعقيد المشروع، وضوح المتطلبات، التقنيات المستخدمة، ضغط الوقت والميزانية.

لذلك يمكن اعتبار ان عملية التطوير الجيدة تزيد احتمال الحصول على منتج عالي الجودة، لكنها ليست الضمان الوحيد.

ثقافة الجودة (Quality Culture)

هي شعور كل فرد في الفريق أن الجودة مسؤوليته الشخصية وليست مسؤولية قسم الجودة فقط فالمبرمج يراجع كوده قبل إرساله و tester يبلغ عن العيوب بوضوح والمحلل يكتب متطلبات دقيقة ومدير المشروع يشجع الالتزام بالمعايير.

▪ عندما تصبح الجودة جزءاً من طريقة التفكير اليومية، تتحول من نشاط منفصل إلى ثقافة يعيشها الفريق.

إذا كنت مدير مشروع، وقلت لفريقك: "أريد برنامجاً عالي الجودة" هل هذه الجملة وحدها تكفي؟

حتماً لا لأن الجميع قد يفسر كلمة "جودة" بطريقة مختلفة لذلك ظهرت فكرة مهمة جداً في هندسة البرمجيات وهي:

ضمان الجودة (Quality Assurance - QA)

لنميز الاختلاف بين المفهومين التاليين

- ضمان الجودة (Quality Assurance - QA) : هو مجموعة الأنشطة المخططة والمنظمة التي نقوم بها أثناء المشروع لمنع ظهور الأخطاء من البداية و التي تمنح الثقة بأن المنتج سيلتزم بالمتطلبات المطلوبة.
- مراقبة الجودة (Quality Control - QC) : التركيز على اكتشاف الأخطاء في المنتج النهائي

مثال: تخيلوا مصنع سيارات إذا كان خط الإنتاج نفسه مليئاً بالمشكلات وبالتالي احتمالية أن تكون السيارات ممتازة معدومة

الأمر نفسه في البرمجيات إذا كانت عملية التطوير فوضوية فسيظهر ذلك في المنتج النهائي.

أهداف ضمان الجودة

1. تسليم المشروع في الوقت المحدد (On Time)
2. الالتزام بالميزانية (Within Budget)
3. تحقيق متطلبات العميل أو تجاوزها (Meet or Exceed Customer Requirements)

مستويات إدارة الجودة

1. المستوى التنظيمي (Organizational Level) : في هذا المستوى تقوم الشركة بوضع:

- معايير البرمجة.
- سياسات الجودة.
- نماذج التوثيق.
- إجراءات المراجعة.
- آليات الاختبار.

أي أنها تبني الإطار العام الذي ستلتزم به جميع المشاريع.

2. مستوى المشروع (Project Level) : يمتلك كل مشروع خطة جودة (Quality Plan) خاصة به وهذه الخطة تحدد:

- أهداف الجودة.
- الأنشطة المطلوبة.
- المعايير التي سيستخدمها

أي أنها تترجم سياسات الشركة إلى إجراءات تناسب هذا المشروع.

3. مستوى العملية (Process Level) : يتم التركيز على التنفيذ اليومي أي التأكد من أن الفريق يطبق ما تم الاتفاق عليه.

مثال

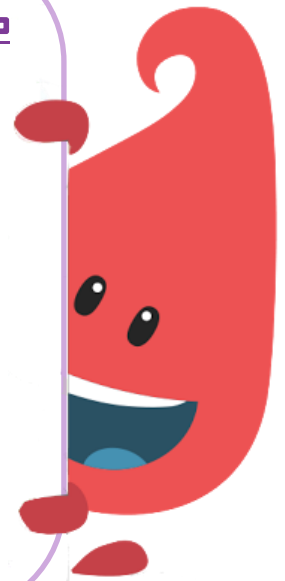
- إذا كانت الخطة تنص أنه يجب مراجعة كل ملف برمجي، يقوم QA بالتأكد أن ذلك يحدث فعلاً
 - إذا كانت الخطة تنص أنه يجب تنفيذ اختبارات الوحدة (Unit Testing)، يتأكد QA من تنفيذها.
- إذن هذا المستوى يهتم بالتطبيق العملي.

هل تختلف إدارة الجودة حسب حجم المشروع؟

- **في المشاريع الضخمة:** نحتاج إلى وثائق كثيرة، خطط جودة مفصلة، تقارير متابعة، سجلات مراجعة وتوثيق كامل لأن أعضاء الفريق قد يتغيرون أثناء المشروع وقد يستمر المشروع سنوات، فالتوثيق ضروري للحفاظ على استمرارية العمل.
- **في المشاريع الصغيرة:** الإفراط في التوثيق قد يستهلك وقتاً أكثر من فائدته لذلك بدلاً من إنتاج عشرات الوثائق يكون التركيز أكبر على: التعاون، الالتزام بالممارسات الجيدة ونشر ثقافة الجودة.
- **كيف أساليب إدارة الجودة عند استخدام التطوير الرشيق (Agile Development) كون Agile يعتمد على:**
 - دورات تطوير قصيرة.
 - تسليم مستمر.
 - تقليل التوثيق.
 - التعاون المباشر.

مثال: لنفترض أن شركة تطور نظاماً لإدارة المطارات.

- **المستوى التنظيمي:** يحدد معايير البرمجة، وسياسات المراجعة، وقوائم التوثيق لجميع المشاريع.
- **مستوى المشروع:** يضع خطة جودة خاصة بنظام المطارات، مع التركيز على الاعتمادية والأمان.
- **مستوى العملية:** يتأكد يومياً من تنفيذ مراجعات الكود، واختبارات الوحدة (Unit Testing)، والاختبارات التكاملية (Integration Testing) وفق الخطة، بهذه الطريقة تتحول الجودة من فكرة عامة إلى ممارسات تنفذ طوال دورة حياة المشروع.



مكونات خطة الجودة

1. تحديد Desired Quality Attributes ضمن process
2. التوقيت Timing : كل نشاط جودة يجب أن يكون له توقيت واضح.
3. تحديد Type of QA Activity
4. الجهة المسؤولة عن تطبيق النشاط وتكون إحدى ثلاث جهات:
 - الفريق البرمجي المسؤول عن تطوير المنتج.
 - موظفي إدارة الجودة.
 - هيئات مستقلة خارج المشروع، يتم الاتفاق معها (لأن الشخص الذي كتب الكود قد لا يرى أخطاءه بسهولة بينما الشخص الخارجي ينظر بعين محايدة).

أنواع أنشطة الجودة.

- مراجعات التصميم (Design Reviews)
- المراجعات الفنية (Technical Reviews)
- الفحص (Inspection)
- الاختبارات (Testing)
- التدقيق (Audit)

بعد أن تعرفنا على مفهوم خطة الجودة وأهميتها، ننتقل الآن إلى مكوناتها الأساسية.

Quality Plan structure

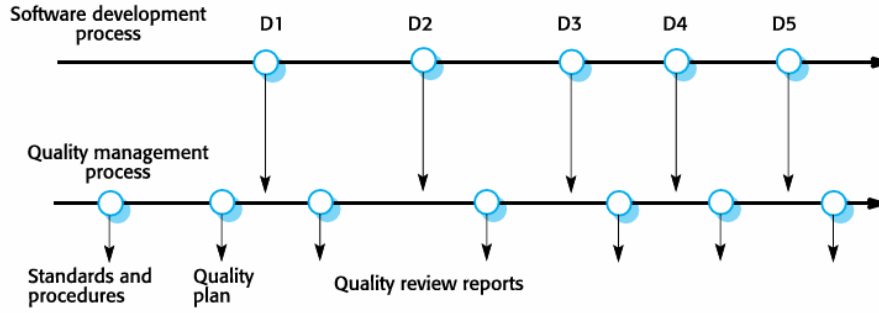
- Product introduction: يقوم بشرح النظام و هدفه و مستخدمين النظام و نطاق المشروع
- Product plans: تتضمن كيفية تطوير النظام و المراحل و الجداول الزمنية و الأنشطة الأساسية
- Process descriptions: شرح العمليات التي سيتم اتباعها.
- Quality goals: تحدد الأهداف القابلة للقياس.
- Risks and risk management: معرفة ما يمنع من الوصول للجودة المطلوبة

قد يكون الخطر:

- نقص الخبرة.
- ضغط الوقت.
- تغيير المتطلبات.
- نقص المختبرين.
- استخدام تقنية

ملاحظة: يجب أن تكون خطة الجودة قصيرة ومختصرة لأن الوثيقة الطويلة غالباً لا يقرأها أحد.

- بعض الفرق تكتب خطة من 200 صفحة ثم تبقى في الأرشيف بينما الخطة الجيدة هي التي يستطيع الفريق الرجوع إليها بسهولة أثناء العمل، إذاً الهدف من الوثيقة ليس عدد الصفحات بل سهولة استخدامها.
- يكون مبدأ مزامنة ال process plan مع ال quality plan كالتالي:



فقبل البدء وإنتاج milestones يتم وضع خطة ال quality plan ثم بعد إنتاج كل work product يتم فحص مدى جودته وتحقيق سمات الجودة المطلوبة ثم يتم رفع report خاص بالمرحلة والنتائج التي تم الوصول اليها في هذه المرحلة

أمثلة على أنشطة الجودة يمكن إدراجها في الخطة.

مثل:



- مراجعة المتطلبات (Requirements Review)
- مراجعة التصميم (Design Review)
- الفحص (Inspection)
- اختبار الوحدة (Unit Testing)
- اختبار النظام (System Testing)
- مراجعة دليل المستخدم (User Manual Review)

مثال: لنفترض أن فريقاً يطور تطبيقاً للخدمات البنكية قد تتضمن خطة الجودة:

- خاصية الجودة: الاعتمادية (Reliability) .
 - الهدف: ألا تتجاوز نسبة الأعطال 0.1% .
 - النشاط: مراجعة التصميم قبل البرمجة، ثم اختبار الوحدة (Unit Testing) ، ثم اختبار النظام (System Testing)
 - التوقيت: كل مرحلة في وقتها المناسب.
 - المسؤول: فريق التطوير، مع مراجعة نهائية من فريق الجودة.
- بهذه الصورة تصبح الجودة جزءاً من خطة المشروع، وليست نشاطاً يتم التفكير فيه بعد الانتهاء من البرمجة.

النموذج الكمي لتخطيط الجودة (Software Quality Planning Model)

كل نشاط جودة يستهلك وقتاً وأشخاصاً وميزانية لذلك لا بد من وجود طريقة علمية تساعدنا على اختيار أنشطة الجودة المناسبة ومن هنا ظهر هذا النموذج.

يهتم النموذج بقياس شيئين أساسيين.

1. فعالية إزالة العيوب (Defect Removal Effectiveness): أي عدد الأخطاء التي نجحنا في اكتشافها وإصلاحها.
2. تكلفة إزالة العيوب (Cost of Defect Removal): أي تكلفة إصلاح الأخطاء.

يعتمد النموذج SQ على ثلاثة أنواع من البيانات

1. توزيع العيوب (Defect Origin Distribution): تحديد المرحلة التي ظهرت فيها الأخطاء

مثال: إذا كتب محلل النظام مطلباً خاطئاً ثم انتقل هذا الخطأ إلى التصميم ثم إلى البرمجة ثم اكتشفناه أثناء اختبار النظام.

نلاحظ أن الخطأ نشأ في مرحلة: المتطلبات (Requirements) وليس في مرحلة الاختبار.

توزيع الأخطاء داخل المشروع في الجدول التالي

المرحلة	نسبة الأخطاء
المتطلبات (Requirements)	15%
التصميم (Design)	35%
البرمجة والدمج (Coding / Integration)	40%
التوثيق (Documentation)	10%

ملاحظة: إن أكبر نسبة من الأخطاء تظهر عادة أثناء البرمجة والدمج لكن هذا لا يعني أن Coding هي الأخطر دائماً لأن خطأ واحداً في المتطلبات قد يولد عشرات الأخطاء لاحقاً ، لهذا لا يجوز إهمال المراحل الأولى.

2. فعالية إزالة العيوب (Defect Removal Effectiveness): تقيس نسبة الأخطاء التي أزيلت ، وليس فقط التي اكتشفت.

- ان عدد الأخطاء التي تتم إزالتها أقل من عدد الأخطاء التي يتم اكتشافها بسبب:
- قد يكون الإصلاح غير كامل.

مثال: وجدنا مشكلة في تسجيل الدخول قام المبرمج بتعديل جزء من الكود لكن بقي سبب آخر يؤدي لنفس المشكلة إذن الخطأ لم يُزل بالكامل.

- قد تظهر أخطاء جديدة أثناء الإصلاح وهذا ما يسمى أحياناً: Regression Defects أي أن إصلاح خطأ قد يؤدي إلى ظهور خطأ آخر.
- بعض الأخطاء لا تكتشف أصلاً فتنتقل إلى المرحلة التالية ثم تتراكم مع الأخطاء الجديدة.

مثال: لنفترض أن مرحلة التصميم أنتجت: 20 خطأ ، خلال مراجعة التصميم اكتشف الفريق: 15 خطأ.

لكن بعد الإصلاح تبين أن 12 فقط أزيلت بالكامل وثلاثة بقيت موجودة بسبب إصلاح غير صحيح إذن:

■ Detection = 15

■ Removal = 12

3. تكلفة إزالة العيوب (Cost of Defect Removal) : إن تكلفة الإصلاح ترتفع كلما تأخر اكتشاف الخطأ.

مثال: تخيلوا بناء منزل إذا اكتشف المهندس أثناء الرسم أن مكان الباب غير مناسب فسيغيره خلال دقيقة أما إذا اكتشف ذلك بعد بناء المنزل فقد يضطر إلى هدم الجدار وإعادة البناء وتعديل الكهرباء وتعديل الدهان.

❧ قبل تشغيل البرنامج، كيف نستطيع اكتشاف الأخطاء؟

من خلال استخدام (Reviews) و (Inspections) وهذه تعتبر من تقنيات (Static Verification) لأنها لا تحتاج إلى تشغيل البرنامج.

▪ Review : هي عملية يقوم فيها مجموعة من الأشخاص بفحص جزء من النظام أو وثائقه بهدف اكتشاف المشكلات المحتملة.

يمكن مراجعة:

- وثيقة المتطلبات (Requirements Specification)
- التصميم (Design)
- الكود البرمجي (Source Code)
- خطة الاختبار (Test Plan)
- دليل المستخدم (User Manual)
- معايير البرمجة (Coding Standards)

أي أن أي مستند أو منتج داخل المشروع يمكن مراجعته.

إن المنتج قد يحصل بعد المراجعة على ما يسمى: Sign-off أي الموافقة الرسمية على الانتقال إلى المرحلة التالية.

❧ أنواع المراجعات (Types of Review)

1. المراجعة الرسمية (Formal Review)

تقييم التقدم الحاصل للمنتج ويملك صلاحية الموافقة على الانتقال بين المراحل أي أنها لا تركز فقط على اكتشاف الأخطاء، بل أيضاً على اتخاذ قرار إداري.

مثال: بعد انتهاء تصميم النظام يعقد اجتماع، إذا وافق فريق المراجعة ينتقل المشروع إلى مرحلة البرمجة.

2. الفحص (Inspection)

يتم استخدامه بعد الانتهاء من المنتج بشكل كامل، يتم فيه اكتشاف الأخطاء والانحراف عن المعايير دون اتخاذ قرار إداري (ليس رسمي)

مثال: لدينا ملف برمجي نجتمع لقراءته سطرًا سطرًا نكتب جميع الأخطاء الموجودة ثم ينتهي الاجتماع، لم نتخذ قراراً بإكمال المشروع أو إيقافه كل ما فعلناه هو اكتشاف العيوب.



كل الصعاب بلطف الله هيّنة...

دورة حياة المراجعة الرسمية (Formal Review Process)

1. (Pre-review Activity): التحضيرات اللازمة لعمل ال review وجعلها عملية فعالة وسهلة مثل:

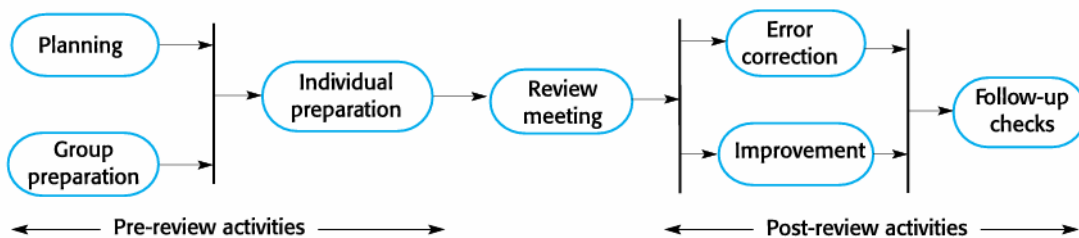
- اختيار المكان المناسب للاجتماع
- تحديد الوقت المناسب
- قراءة ال documentation الخاص بال process أو ال product المراد فحصه
- عقد اجتماع اولي بسيط لتحديد الأفكار المراد مناقشتها

2. اجتماع المراجعة (Review Meeting): يتم تقديم presentation من قبل الفريق المطور للعمل المقدم للفحص ويتم دراسة وفحص العمل من قبل فريق review ومن ثم توثيق نتيجة الاجتماع بمستند لتقييم المرحلة

3. ما بعد المراجعة (Post-review Activities): بعد انتهاء الاجتماع لا تنتهي العملية بل يبدأ العمل الحقيقي، أي أن اجتماع المراجعة ليس النهاية بل بداية عملية التحسين ليقوم الفريق المطور بـ:

- إصلاح الأخطاء.
- تعديل الوثائق.
- تنفيذ الإجراءات المطلوبة.
- التأكد من معالجة جميع الملاحظات.

ويكون مخطط ال formal review كالتالي:



المراجعات عن بُعد (Remote reviewing)

تتم باستخدام:

- مستندات مشتركة.
 - أدوات التعليق (Annotations).
 - أنظمة إدارة الكود مثل Git.
 - منصات التعاون مثل GitHub أو GitLab أو Azure DevOps.
- كل عضو يقرأ الوثيقة ثم يضيف تعليقاته ثم يناقش الفريق هذه الملاحظات عبر اجتماع افتراضي.

إذا لم تعد عملية Review مرتبطة بوجود جميع أعضاء الفريق في المكان نفسه، إذ يمكن تنفيذها عن بُعد باستخدام أدوات التعاون المختلفة.

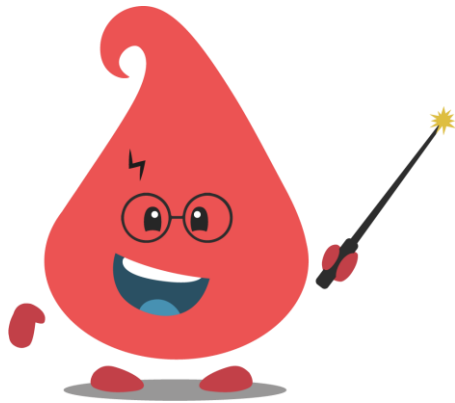
تعتمد المراجعات في هندسة البرمجيات على توزيع واضح للأدوار وكل شخص يعرف ماذا يفعل ومتى يتحدث، وما هي مسؤوليته.

المشاركون في المراجعة (Review Participants)

1. قائد المراجعة (Review Leader)
2. فريق المراجعة (Review Team)
3. فريق التطوير (Development Team)

أولاً: قائد المراجعة (Review Leader)

- يجب أن يكون خارج فريق المشروع لإصدار حكم موضوعي للدفاع عن الجودة.
- يمتلك خبرة في نوع المراجعة.
- درجته الوظيفية مناسبة أي لا يكون أقل خبرة من قائد المشروع لأنه قد يضطر أحياناً إلى رفض التصميم بالكامل
- العلاقة الجيدة مع الفريق.



مسؤوليات قائد المراجعة

- تنظيم الاجتماع.
- إدارة النقاش.
- إعطاء كل شخص فرصة للكلام.
- منع الخروج عن موضوع الاجتماع.
- تلخيص النتائج.
- التأكد من تسجيل جميع الملاحظات.

ثانياً: فريق المراجعة (Review Team)

- هم الأعضاء ذوي الخبرة لأن الهدف ليس قراءة الوثيقة فقط بل تحليلها واكتشاف الأخطاء الخفية وهذا يحتاج إلى خبرة عملية.
- عدد أعضاء الفريق من 3 إلى 5 أعضاء.

- ثالثاً: فريق التطوير (Development Team) : يقدم عرضاً مختصراً عن الوثيقة أو التصميم ليس مطلوباً أن يقرأ الوثيقة من البداية للنهاية.

بل يركز على الفكرة العامة، القرارات المهمة والنقاط التي تحتاج إلى توضيح.

خطوات الاجتماع:

1. يفتتح قائد المراجعة الاجتماع ويقوم بشرح:
 - الهدف
 - جدول الأعمال
 - الوثيقة التي ستراجع

2. يعرض فريق التطوير الوثيقة ويركز على أهم القرارات والأفكار.
3. يبدأ أعضاء فريق المراجعة بطرح ملاحظاتهم.
4. يناقش الجميع الإجراءات المطلوبة.

القرارات التي قد تصدر عن الاجتماع

1. **الموافقة الكاملة (Full Approval):** أي الوثيقة جيدة قد توجد بعض الملاحظات البسيطة لكن المشروع يستطيع الانتقال مباشرة إلى المرحلة التالية.
 2. **الموافقة الجزئية (Partial Approval):** بعض أجزاء المشروع جاهزة أما أجزاء أخرى فتحتاج إلى تعديلات كبيرة. لذلك يسمح للفريق بالاستمرار في الأجزاء الجاهزة فقط أما بقية الأجزاء فلا تستكمل إلا بعد تنفيذ التعديلات المطلوبة.
 3. **رفض الموافقة (Denial of Approval):** بسبب احتواء الوثيقة على أخطاء كبيرة أو أخطاء حرجية (Critical Defects).
- وبالتالي يجب إعادة المراجعة بعد إصلاح المشكلات أي أن المشروع لا ينتقل إلى المرحلة التالية.

مثال: انتهى فريق من تصميم نظام لإدارة مطار بعد اجتماع المراجعة:

- وجد الفريق أن معظم التصميم صحيح لكن وحدة إدارة الرحلات تحتوي على أخطاء في معالجة الحالات الاستثنائية.
- لذلك صدر قرار Partial Approval:
- يسمح بمتابعة العمل في بقية الوحدات بينما تُعاد مراجعة وحدة إدارة الرحلات بعد تصحيح تصميمها.

الفحص (Software Inspection)

هو عملية مراجعة رسمية ومنظمة تهدف إلى اكتشاف العيوب في المنتج البرمجي قبل تشغيله ويعتمد (Inspection) على:

- خطوات واضحة.
- أدوار محددة.
- قوائم تحقق (Checklists).

أي لا يمكن الاكتفاء ب testing لان الفحص يكمل الاختبارات، ولا يحل محلها.

قوائم التحقق (Checklists)

- هي قائمة تحتوي على أسئلة أو نقاط يجب التأكد منها أثناء الفحص بدلاً من الاعتماد على الذاكرة.
- قائمة التحقق ليست ثابتة بل تتطور مع مرور الوقت.
- تستخدم لتحقيق عدة فوائد منها:
 1. تقليل احتمال نسيان نقطة مهمة.
 2. جعل جميع أعضاء الفريق يقومون بال Review بالطريقة نفسها.
 3. زيادة جودة المراجعة لأنها مبنية على خبرات المشاريع السابقة.

Fault class	Inspection check
Interface faults	- Do all function and method calls have the correct number of parameters? - Do formal and actual parameter types match? - Are the parameters in the right order? - If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	- If a linked structure is modified, have all links been correctly reassigned? - If dynamic storage is used, has space been allocated correctly? - Is space explicitly deallocated after it is no longer required?
Exception management faults	Have all possible error conditions been taken into account?

Fault class	Inspection check
Data faults	- Are all program variables initialized before their values are used? - Have all constants been named? - Should the upper bound of arrays be equal to the size of the array or Size -1? - If character strings are used, is a delimiter explicitly assigned? - Is there any possibility of buffer overflow?
Control faults	- For each conditional statement, is the condition correct? - Is each loop certain to terminate? - Are compound statements correctly bracketed? - In case statements, are all possible cases accounted for? - If a break is required after each case in case statements, has it been included?
Input/output faults	- Are all input variables used? - Are all output variables assigned a value before they are output? - Can unexpected inputs cause corruption?

ملاحظة: لأجل تحسين عملية الفحص قد يكتشف الفريق أن الاجتماعات طويلة جداً وقوائم التحقق أصبحت كبيرة أكثر من اللازم و بعض البنود لم تعد مفيدة، فيقوم بتعديل العملية وهذا ما يسمى التحسين المستمر (Continuous Improvement).

ال جودة في التطوير الرشيق (Agile Development)

- Agile يهتم بالجودة منذ اليوم الأول لكن بأسلوب مختلف عن المشاريع التقليدية.
- في منهجيات Agile لا تعتمد إدارة الجودة على الوثائق الكثيرة والإجراءات الرسمية، وإنما تعتمد على نشر ثقافة الجودة داخل الفريق بحيث يشعر كل عضو بأنه مسؤول عن جودة البرنامج، مع تجنب الإجراءات البيروقراطية الثقيلة.

المنهجيات التقليدية (Traditional Development) : تتم المراجعات والاختبارات في مراحل منفصلة.

Agile: الجودة تصبح جزءاً من كل دورة تطوير (Iteration) أي أنه في نهاية كل Sprint يتم عادة:

- مراجعة الكود.
- تنفيذ اختبارات الوحدة (Unit Testing).
- تنفيذ اختبارات التكامل (Integration Testing).
- مراجعة التغييرات.
- إصلاح الأخطاء مباشرة.

مبادئ الجودة في Agile

- Check before check-in:** يقوم المبرمج بمراجعة الكود مع أحد أعضاء الفريق قبل رفعه إلى مستودع المشروع
- Never break the build:** يجب ألا يرفع المطور كوداً يؤدي إلى فشل بناء النظام، لذلك يجب اختبار التعديلات قبل دمجها.

- **Fix problems when you see them**: إذا اكتشف أحد المطورين مشكلة في كود كتبه شخص آخر، فمن الأفضل إصلاحها مباشرة بدلاً من تأجيلها.
- ويتناسب ذلك مع Agile ولو انتظر الفريق حتى نهاية المشروع لاكتشاف الأخطاء، فسيفقد أهم ميزة في Agile، وهي (Fast Feedback)

المراجعات في Agile

- تكون المراجعات غير رسمية مقارنة بالمشاريع التقليدية .
- في Scrum يتم عقد Sprint Review بعد نهاية كل Sprint لمناقشة جودة المنتج والمشكلات .
- في Extreme Programming يحقق Pair Programming مراجعة مستمرة للكود لأن كل مبرمج يعمل مع زميله .



Be proud of yourself for fighting so hard to become the person you want to be.